Лабораторна робота №2. Створення компонентів. Дерево компонентів і передача пропсів

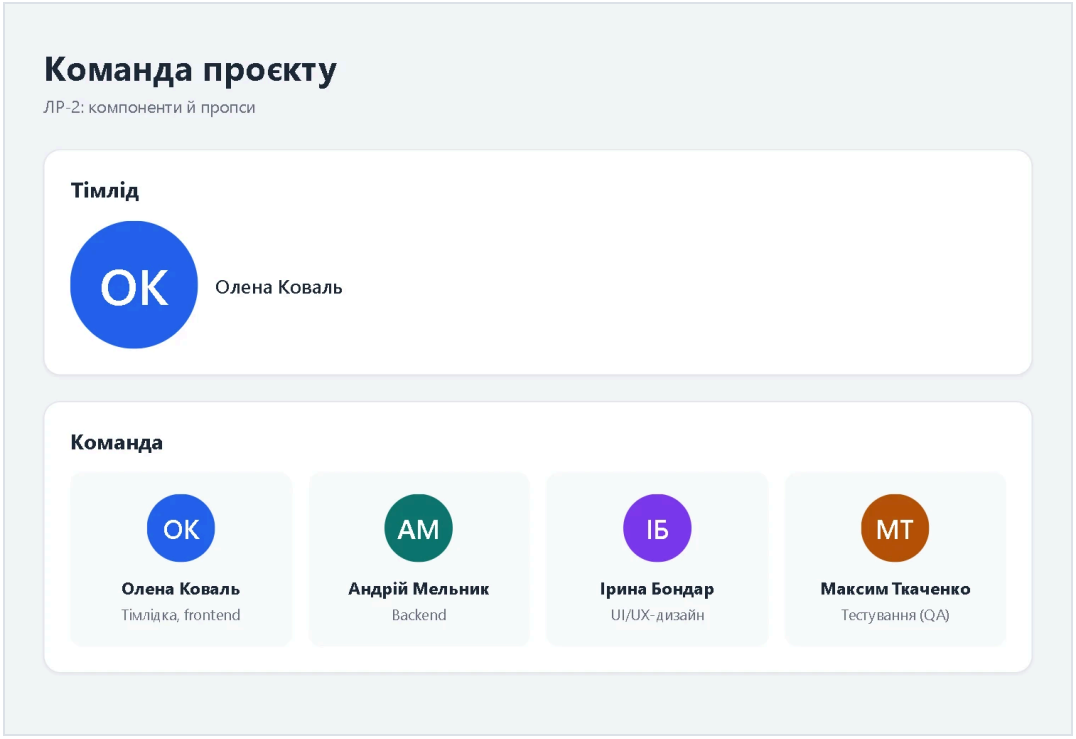
> **Примітка щодо актуальності.** Документ підготовлено станом на 4 листопада 2026 року: React 19, Vite 8, ESLint 10, Prettier 3. У разі відмінностей орієнтуйтеся на документацію React (§2).

> **Якщо ЛР-1 у вас лежить в окремому репозиторії `react-lab-1`,** спершу перенесіть її в репозиторій `react-labs` за ЛР-1, §4.9: проєкт цієї роботи додається в той самий репозиторій.

1. Мета роботи

Написати власноруч чотири React-компоненти, передати їм дані через пропси й побачити застосунок як два дерева: дерево рендерингу й дерево залежностей модулів.

Очікуваний результат: у теці `lab-2/` репозиторію `react-labs` — сторінка «Команда проєкту» з компонентів `Header`, `Avatar`, `Profile` і `Card`; у `lab-2/README.md` — обидва дерева; застосунок відкривається на Vercel.



Компоненти сторінки:

```
App
├── Header
├── Card «Тімлід»
│   └── Avatar           size={120}
└── Card «Команда»
    └── Profile × 4
        └── Avatar       size не передано
```

2. Корисні посилання

Теоретичної частини в цій роботі немає: усе потрібне є в документації React українською. Перед заняттям прочитайте сторінки 4 і 6, решту відкривайте, коли до них відсилає завдання.

1. Ваш перший компонент — компонент як функція, що повертає розмітку; чому його назва починається з великої літери.
2. Імпорт та експорт компонентів — один компонент у файлі, `export default` та `import`.
3. JavaScript у JSX із фігурними дужками — `{…}` в атрибутах і в тексті; чим `"120"` відрізняється від `{120}`.
4. Передача пропсів до компонента — головна сторінка цієї роботи.
5. Рендеринг списків — `map` і `key` (§4.4).
6. Розуміння вашого UI як дерева — дерево рендерингу й дерево залежностей модулів (§4.6).
7. Інструменти розробника React — вкладка ⚛ **Components**.

3. Завдання лабораторної роботи

1. Додати в репозиторій `react-labs` теку `lab-2/` зі стартового проєкту (§4.1).
2. `Header`: рядкові пропси (§4.2).
3. `Avatar`: об'єкт у пропсі й значення за замовчуванням (§4.3).
4. `Profile` і список команди (§4.4).
5. `Card`: `children` (§4.5).
6. Дерево рендерингу й дерево залежностей модулів (§4.6).
7. Перевірки, GitHub і Vercel (§4.7), звіт (§5).

Усі завдання §4.1–4.7 виконуються на занятті (орієнтовно 25–30 хвилин); звіт (§5) можна надіслати після нього. §4.8 — додаткове завдання для тих, хто впорався раніше.

AI. Код цієї роботи пишіть самостійно. AI-автодоповнення в редакторі можна залишити ввімкненим, але кожен запропонований рядок ви маєте вміти пояснити на захисті. Доручати завдання AI-агентові не рекомендовано: компоненти короткі, і мета — набити руку на компонентах і пропсах.

4. Хід виконання роботи

Кожне завдання закінчується рядком «Що має вийти»: перевіряйте сторінку в браузері після кожного кроку. Файли зберігайте як у ЛР-1 (`Ctrl+S`): Prettier відформатує їх під час збереження.

4.1. Стартовий проєкт

1. Відкрийте в редакторі теку `react-labs` і перевірте, що `git status` чистий. Якщо працюєте на іншому комп'ютері, спершу клонуйте `react-labs` з GitHub.
2. Завантажте lab-2.zip і розпакуйте його в провіднику (Windows: **Extract All**, macOS: подвійний клік). Вийде тека `lab-2`. Перемістіть її в корінь `react-labs`, поруч із `lab-1/`. Не розпаковуйте архів командою в терміналі в корені `react-labs`: файли в ньому лежать без теки й замінили б кореневі `package.json` і `.gitignore`.
3. У корені репозиторію виконайте `git status -u`: усі нові файли мають бути в `lab-2/`.
4. Встановіть залежності в корені й запустіть dev-сервер:

   ```
   npm install
   cd lab-2
   npm run dev
   ```

   `npm install` додає `lab-2` у кореневий `package-lock.json`. Відкрийте http://localhost:5173: на сторінці один рядок тексту. Dev-сервер не зупиняйте до §4.7, а для команд git відкрийте другий термінал (у VS Code — **+** на панелі терміналу): він відкривається в корені `react-labs`.
5. Що вже є в проєкті:

Файл	Що в ньому
`src/data.js`	масив `team`: чотири учасники з полями `id`, `name`, `role`, `photo`, `isLead`
`src/index.css`	усі стилі сторінки; у компонентах потрібні лише класи із завдань
`public/avatars/`	зображення аватарів

Файл	Що в ньому
`README.md`	два дерева, які ви заповните в §4.6
`package.json`, `eslint.config.js`, `.prettierrc.json`	скрипти й налаштування ESLint і Prettier, як у ЛР-1

6. Зробіть коміт у корені репозиторію: так до нього потрапить і `package-lock.json`.

```
git add .
git commit -m "chore: add lab 2 starter"
```

Далі шляхи до файлів (`src/App.jsx`) — відносно `lab-2/`.

4.2. `Header`: рядкові пропси

Перший компонент — приклад. Перепишіть його й розберіться в кожному рядку.

1. Створіть теку `src/components/` і в ній файл `Header.jsx`:

```
export default function Header({ title, subtitle }) {
  return (
    <header className="header">
      <h1>{title}</h1>
      <p>{subtitle}</p>
    </header>
  )
}
```

   - Компонент — функція, що повертає JSX. Назва починається з великої літери: так React відрізняє ваш `<Header>` від HTML-тегу `<header>`.
   - React передає всі пропси одним об'єктом — першим параметром функції. `{ title, subtitle }` — деструктуризація: вона дістає з цього об'єкта два поля.
   - `{title}` у розмітці вставляє значення пропа.
   - `export default` дає змогу імпортувати компонент в іншому файлі.

2. Замініть вміст `src/App.jsx`:

```
import Header from './components/Header.jsx'

export default function App() {
  return (
    <>
      <Header title="Команда проєкту" subtitle="ЛР-2: компоненти й пропси"
/>
      <main>
        <p>Тут буде сторінка команди.</p>
      </main>
    </>
  )
}
```

   Компонент повертає один кореневий елемент, тому `Header` і `<main>` загорнуто у фрагмент `<>…</>`: він групує елементи й нічого не додає в DOM.

Що має вийти: заголовок «Команда проєкту» з підзаголовком.

Коміт: `feat: add Header`. **Довідка:** Крок 2: Читайте пропси всередині дочірнього компонента.

4.3. `Avatar`: об'єкт у пропсі й значення за замовчуванням

Далі код компонентів пишете самі. Створіть `src/components/Avatar.jsx`.

Проп	Що це	За замовчуванням
`person`	об'єкт учасника з `data.js`	—
`size`	ширина й висота зображення в пікселях	`64`

Компонент рендерить один `<img>` з `className="avatar"` і такими атрибутами:

Атрибут	Значення
`src`	`person.photo`
`alt`	`person.name`
`width`, `height`	`size`

Значення за замовчуванням задають у деструктуризації: `{ person, size = 64 }`.

Щоб перевірити компонент, додайте в `App.jsx` два імпорти:

```
import Avatar from './components/Avatar.jsx'
import { team } from './data.js'
```

і замініть рядок … усередині <main> двома аватарами:

```
<Avatar person={team[0]} size={120} />
<Avatar person={team[1]} />
```

Число передають у фігурних дужках: size={120}. Запис size="120" передав би рядок.

Що має вийти: великий аватар «ОК» і менший «АМ». Другому size не передано, тож діє значення за замовчуванням.

Коміт: feat: add Avatar. **Довідка:** Знайомі пропси, Вказання значення за замовчуванням для пропа.

4.4. Profile і список команди

Створіть src/components/Profile.jsx. Проп — person. Компонент рендерить <article className="profile">, а в ньому по черзі:

1. Avatar з пропом person і без size;
2. <h3> з person.name;
3. з person.role.

Profile використовує Avatar, тож імпортуйте його у Profile.jsx: import Avatar from './Avatar.jsx' — шлях відносно теки src/components/.

У App.jsx імпортуйте Profile:

```
import Profile from './components/Profile.jsx'
```

і замініть другий аватар (той, що без size) списком команди. Перший аватар залиште: у §4.5 він переїде в картку «Тімлід».

```
{team.map((person) => (
  <Profile key={person.id} person={person} />
))}
```

map перетворює кожного учасника на <Profile>. key допомагає React розрізняти елементи списку; його беруть з даних (id), а не з індексу масиву.

Що має вийти: великий аватар, під ним — чотири профілі з аватарами, іменами й ролями.

Коміт: feat: add Profile and the team list. **Довідка:** Крок 1: Передайте пропси до дочірнього компонента, Збереження порядку елементів списку за допомогою key.

4.5. `Card`: `children`

Створіть `src/components/Card.jsx`.

Проп	Що це
`title`	заголовок картки
`children`	усе, що стоїть між `<Card>` і `</Card>`

Компонент рендерить:

```
<section className="card">
  <h2>{title}</h2>
  <div className="card-content">{children}</div>
</section>
```

Тепер зберіть сторінку: остаточний `src/App.jsx` має бути таким.

```
import Header from './components/Header.jsx'
import Avatar from './components/Avatar.jsx'
import Profile from './components/Profile.jsx'
import Card from './components/Card.jsx'
import { team } from './data.js'

export default function App() {
  const lead = team[0]

  return (
    <>
      <Header title="Команда проєкту" subtitle="ЛР-2: компоненти й пропси" />
      <main>
        <Card title="Тімлід">
          <Avatar person={lead} size={120} />
          <p>{lead.name}</p>
        </Card>
        <Card title="Команда">
          {team.map((person) => (
            <Profile key={person.id} person={person} />
          ))}
        </Card>
      </main>
    </>
  )
}
```

Той самий `Card` показує різний вміст: JSX, вкладений між `<Card>` і `</Card>`, React передає в проп `children`.

Що має вийти: сторінка, як на знімку в §1.

Коміт: `feat: add Card`. **Довідка:** Передача JSX як children.

4.6. Два дерева

1. Відкрийте DevTools → ⚛ **Components** і розгорніть дерево. Порівняйте його зі схемою в §1. Біля кожного `Profile` DevTools показує його `key`.
2. Виберіть `Avatar` у картці «Тімлід», потім `Avatar` у будь-якому `Profile`, і порівняйте їхні **props**. Чому `size` є лише в першого, хоча другий теж має розмір, — запитають на захисті.
3. У того самого `Avatar` з картки «Тімлід» клацніть значення `size` у **props**, введіть `200` і натисніть `Enter`: аватар збільшиться без зміни коду. Перезавантажте сторінку — розмір знову 120.
4. Для цього ж `Avatar` знайдіть поле **rendered by**: хто його рендерить?
5. Заповніть у `lab-2/README.md` обидва дерева:
   - **дерево рендерингу** — за вкладкою ⚛ Components: усі компоненти від `App` до кожного `Avatar`, з усіма чотирма `Profile`;
   - **дерево залежностей модулів** — за рядками `import`: від `main.jsx` до кожного файлу, який імпортується прямо чи через інший файл, разом з `index.css` і `data.js`; пакети `react` і `react-dom` не потрібні.

   Пишіть у стилі схеми з §1: символи ├──, └── і │ можна скопіювати звідти, а можна позначати вкладеність лише відступами.

Коміт: `docs: add the lab 2 trees`. **Довідка:** Дерево рендерингу, Дерево залежностей модуля.

4.7. Перевірки, GitHub і Vercel

1. Зупиніть dev-сервер (`Ctrl+C`) і виконайте в `lab-2/`:

   ```
   npm run lint
   npm run format:check
   npm run build
   ```

   Усі три команди мають пройти без помилок. Якщо `format:check` назве файл, відкрийте його, збережіть ще раз і закомітьте: `style: format lab 2 files`.
2. Відправте коміти на GitHub: `git push`. Історія комітів цієї роботи:
   1. `chore: add lab 2 starter`;
   2. `feat: add Header`;
   3. `feat: add Avatar`;

4. `feat: add Profile and the team list`;
5. `feat: add Card`;
6. `docs: add the lab 2 trees`.

3. На vercel.com → **Add New** → **Project** ще раз імпортуйте `react-labs`: це буде другий проєкт з того самого репозиторію. Перед **Deploy** налаштуйте:

Поле	Значення
Project Name	`react-lab-2`
Root Directory	**Edit** → `lab-2`
Framework Preset	Vite (Vercel розпізнає його сам)
Output Directory	`dist`

4. Після деплою скопіюйте адресу застосунку з вкладки **Overview** проєкту, поле **Domains**, і відкрийте її: має відкритися сторінка «Команда проєкту».

4.8. Додаткове завдання

Не обов'язкове й не оцінюється. Створіть `src/components/Badge.jsx`: компонент рендерить `<span className="badge">` з `children` усередині. У `Profile` під роллю додайте:

```
{person.isLead && <Badge>Тімлід</Badge>}
```

`Badge` з'явиться лише в профілі Олени. Знайдіть його у вкладці ⚛ Components: дерево рендерингу тепер залежить від даних. Допишіть `Badge` в обидва дерева в `README.md` і закомітьте: `feat: add a badge for the team lead`.

5. Вимоги до звіту

Тема листа:

```
[React] ЛР-2: Компоненти та пропси
```

У листі вкажіть теку лабораторної на GitHub і адресу застосунку на Vercel:

```
GitHub repository: https://github.com/<username>/react-labs/tree/main/lab-2
Vercel: https://<project>.vercel.app/
```

Звіт повинен містити:

1. знімок екрана сторінки з відкритою вкладкою ⚛ **Components**, де вибрано `Avatar` з картки «Тімлід» і видно його props;
2. знімок екрана `npm run lint`, `npm run format:check` і `npm run build` без помилок;

3. чи використовували AI-агента і для чого;
4. короткий висновок: що зроблено й що було складним.

Дерева лежать у `lab-2/README.md`, у звіт можна продублювати.

6. Усний захист

Формат здачі передбачає усну співбесіду за результатами роботи. Питання нижче — типові питання співбесіди frontend-розробника з тем цієї роботи. Відповідайте на прикладах власного коду. Викладач може попросити показати відповідь у коді чи DevTools або змінити код наживо.

Питання для підготовки:

Компоненти

- Що таке компонент у React? Чим він відрізняється від звичайної функції JavaScript?
- Чому назва компонента починається з великої літери? Чим для React відрізняються `<header>` і `<Header>`?
- Чому компоненти не оголошують усередині інших компонентів?
- Чим `export default` відрізняється від іменованого експорту? Як імпортують кожен із них?
- Навіщо тримати кожен компонент в окремому файлі?
- Навіщо фрагмент `<>…</>` в `App`? Що може повернути компонент?

Пропси

- Що таке пропси? Як батьківський компонент передає їх дочірньому?
- Чим `size="120"` відрізняється від `size={120}`?
- Навіщо деструктуризація `{ person, size }` у параметрі? Як прочитати пропси без неї?
- Які значення можна передати через пропси?
- Чим пропси `src` і `alt` елемента `<img>` відрізняються від пропсів вашого `Avatar`?
- Коли діє значення за замовчуванням `size = 64`? Що буде з `size={null}` і `size={0}`?
- Чому в DevTools в `Avatar` усередині `Profile` немає пропа `size`, хоча зображення має розмір 64?
- Чи може компонент змінити свої пропси? Що використовують, коли дані мають змінюватися?
- Що робить запис `<Avatar {...props} />`? Коли він доречний, а коли заважає?
- Що станеться, якщо не передати `person` в `Avatar`?

children

- Що таке `children`? Звідки береться його значення в `Card`?
- Коли краще `children`, а коли окремий проп, як `title`?

Списки й key

- Навіщо `key` у `team.map(...)`? Чому `person.id`, а не індекс масиву?
- Чому `key` стоїть на `<Profile>`, а не всередині `Profile`? Чи може компонент прочитати власний `key`?

Дерева

- Що таке дерево рендерингу і як воно пов'язане з вкладкою ⚛ Components?
- Чим дерево рендерингу відрізняється від дерева залежностей модулів? Чому `Avatar` у першому трапляється п'ять разів, а в другому — один?
- Які компоненти у вашому застосунку кореневі, а які — листки? Чому це важливо для продуктивності?
- Хто рендерить `Avatar` у картці «Тімлід»: `App` чи `Card`? Що DevTools показує в полі **rendered by**?
- Як умовний рендеринг змінює дерево рендерингу?
- Навіщо бандлеру дерево залежностей модулів?

DevTools і деплой

- Як знайти компонент у DevTools за елементом сторінки?
- Чому зміна `size` у DevTools зникла після перезавантаження сторінки?
- Як Vercel збирає `lab-2` з того самого репозиторію, що й `lab-1`? Що задає Root Directory?

7. Критерії готовності

Роботу зараховано після захисту (§6); вона оцінюється в 1 бал.

Критерій	Мінімальна вимога
Проєкт	`lab-2/` у репозиторії `react-labs`; у теці `npm run lint`, `npm run format:check` і `npm run build` проходять без помилок
Компоненти	`Header`, `Avatar`, `Profile`, `Card` — функційні компоненти, кожен у своєму файлі в `src/components/`, з `export default`
Пропси	пропси читаються деструктуризацією; `Avatar` має `size = 64`; `Card` рендерить `children`
Список	профілі рендеряться через `team.map` з `key={person.id}`
Дерева	у `lab-2/README.md` — дерево рендерингу й дерево залежностей модулів, які відповідають коду

Критерій	Мінімальна вимога
GitHub	коміти з §4.7
На занятті	усі коміти §4.1–4.7 зроблено під час заняття (дати видно в `git log`)
Vercel	проєкт з Root Directory `lab-2`; адреса показує сторінку з останнього коміту `main`
Звіт	усі пункти §5